



University of
Nottingham

UK | CHINA | MALAYSIA

Software Engineering

Specifications

COMP1003 - Lecture 7

Dr Horia A. Maior



University of
Nottingham

UK | CHINA | MALAYSIA

Software Engineering

Specifications

COMP1003 - Lecture 7

Dr Horia A. Maior



Today

- Lab Feedback
- Software Specifications



University of
Nottingham

UK | CHINA | MALAYSIA

Meme of the week



University of
Nottingham

UK | CHINA | MALAYSIA

Last Lab Feedback



University of
Nottingham

UK | CHINA | MALAYSIA

Things to think about as a team



Time management

- We designed most of the work to be completed in the 2 hour lab session for the skills labs. Use effective team work to complete this in the given timeframe
- There's emphasis in the marking on
 - team coordination
 - the thought that goes into diagrams
 - how well the diagrams relate to each other



How does a good team work together?

- They work together (not always all together, **sometimes in subgroups**)
- They **plan and agree** on an approach before starting
- They **plan and agree** on what they think 'good work' is like
- Later, they **review and discuss** the work everyone is producing
 - and then do more work to shape things up
- They **do not** just divvy up the work, go separate ways, and try to bring it all together on the last day



Features of git

- Ive taught you several features of git that helps teams work together
 - Issues, milestones, boards etc
- A good team is using these to coordinate work
- A good team member is engaging with all of them, **regularly**
- Im going to be looking for evidence that people have pushed content, got involved in discussions through git, and have used the features.



Getting help in the module

- You can get **some limited** help during the labs
- You can create issues (and assign them to me) in between labs
- But, we cant tell you in advance if e.g. your diagram is good enough 🙄
 - Make use of the slides, recommended reading, the books..



Getting help in the module

- If you do not have 3 active members in your team, please let me know using git issue (assigned to me)
- We are expecting all teams to contribute and do the labs every week.



Asking for help

- I am not the client. Neither are the TAs.
- We cannot answer clarifications on the software brief.
- You are preparing to help your bosses to go to the client with questions/clarifications.



Watch out!

- Dont tag too early.
 - Some groups tagged before they got one of the diagrams in. So the one 'I would mark' is the tagged stage. Not the latest stage.
 - Use good naming for your tags. Do not use UML. Make it clear which lab it was.
- Dont push e.g. word documents/pngs instead of markdown files
 - I am marking markdown reports.
 - In the past, teams just submitted word docs and .vpp files from visual paradigm.
- Make sure your link in the README.md actually works! Test it.
 - Give this job to sometime, and give them time to do it.
 - If you did not link your lab 2 work to the README.md file please do this during the next lab. Add that as your first task.



During Marking or feedback - I will...

- 1) go to your repository
- 2) find the most recent git tag for CW or the LAB
- 3) view the main README as it was at the moment of the git tag.
- 4) look for a link on the README to a markdown file that is your content for CW or LAB
- 5) view and mark the content that is included in that, and any secondary markdown files linked from that primary markdown file.



I wont be...

- Viewing random web links
- Nor non-markdown files in the repository



What makes a good UML Diagram?



What makes a good set of user stories?



What makes a good persona?



What makes a good Scenario?



How do you create integration across diagrams?



What counts as good markdown?



University of
Nottingham

UK | CHINA | MALAYSIA

Software Engineering

Specifications

COMP1003 - Lecture 7

Dr Horia A. Maior



Today's Learning Outcomes

- 0) What specifications are (and what they look like)
- 1) How they are different to requirements
- 2) What makes a good/bad specification
- 3) Who refers to specifications



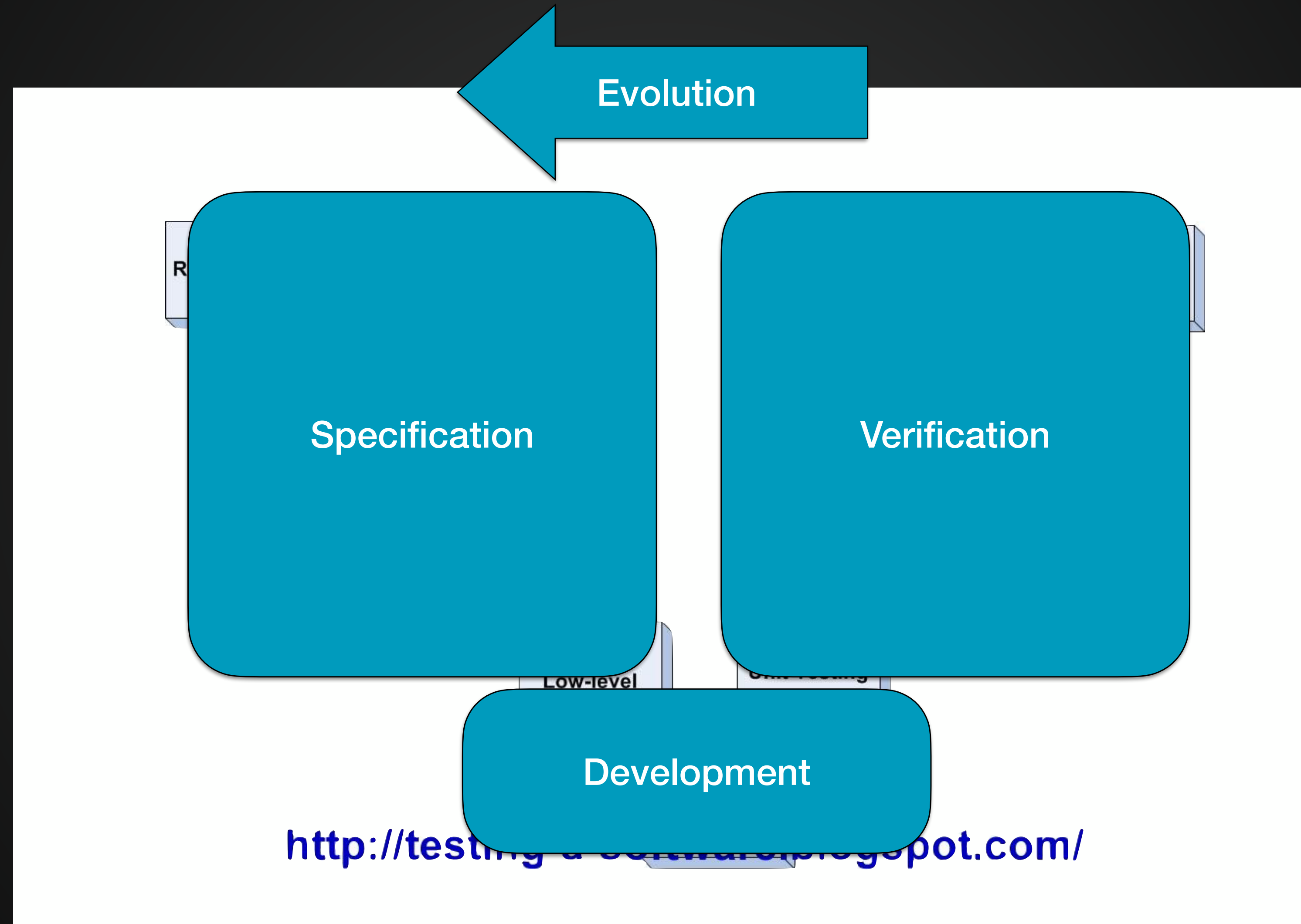
University of
Nottingham

UK | CHINA | MALAYSIA

Where we are in the software
engineering stages?

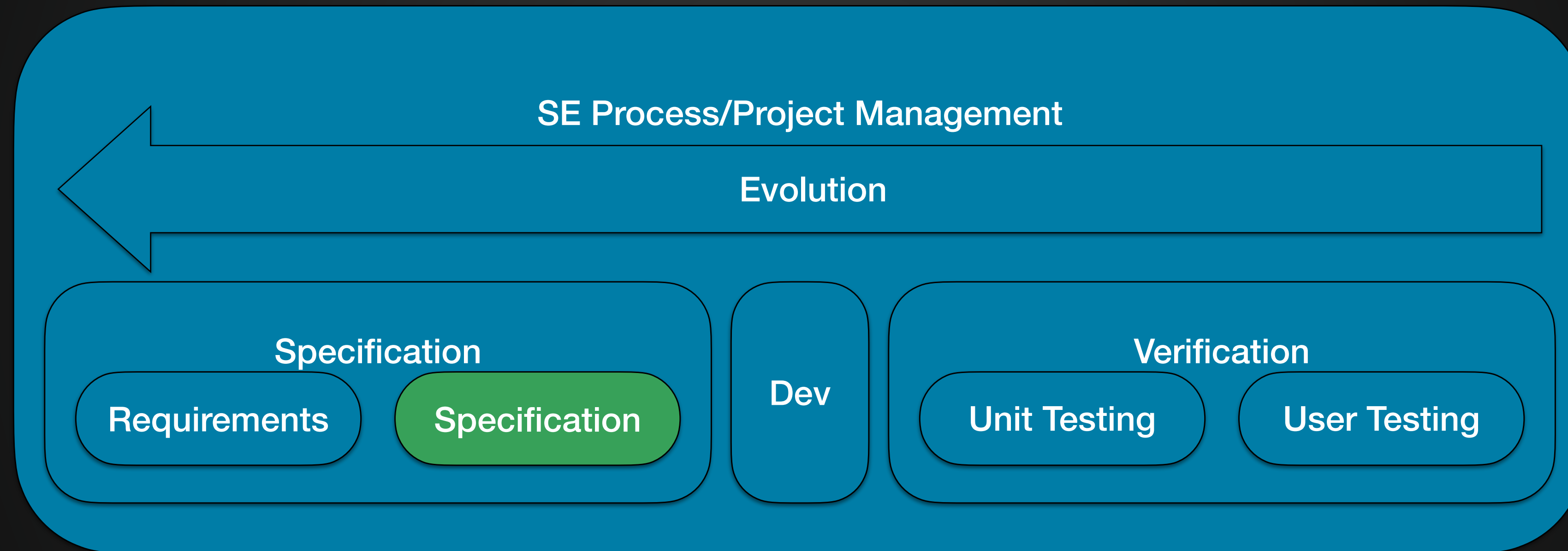


Keeping Track of This Module





Keeping Track of This Module





University of
Nottingham

UK | CHINA | MALAYSIA

Specifications (vs Requirements)



Requirements vs Specifications

	Functional	Non-Functional
Requirements	1) A module convenor will need to see and check if a student has the necessary knowledge to take their module.	
Specifications	1.1) A module interface shall show a list of students who have the necessary knowledge to take their module. 1.2) A module interface shall allow the user to see the module details. 1.3) A module interface shall show additional comments for each student. 1.4) A module interface shall let the user accept/reject a student's request to take the module. 1.5) A module interface shall let the user contact/ask the student to clarify their request.	

What a stakeholder needs to be able to do

What the software must do to meet the requirement above



Requirements vs Specifications

User requirement definition

1. The MHC-PMS shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month.

System ~~requirements~~ specification

- 1.1 On the last working day of each month, a summary of the drugs prescribed, their cost and the prescribing clinics shall be generated.
- 1.2 The system shall automatically generate the report for printing after 17.30 on the last working day of the month.
- 1.3 A report shall be created for each clinic and shall list the individual drug names, the total number of prescriptions, the number of doses prescribed and the total cost of the prescribed drugs.
- 1.4 If drugs are available in different dose units (e.g. 10mg, 20 mg, etc.) separate reports shall be created for each dose unit.
- 1.5 Access to all cost reports shall be restricted to authorized users listed on a management access control list.

Something the stakeholder will need - or a service the software must provide



Requirements vs Specifications

User requirement definition

1. The MHC-PMS shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month.

System ~~requirements~~ specification

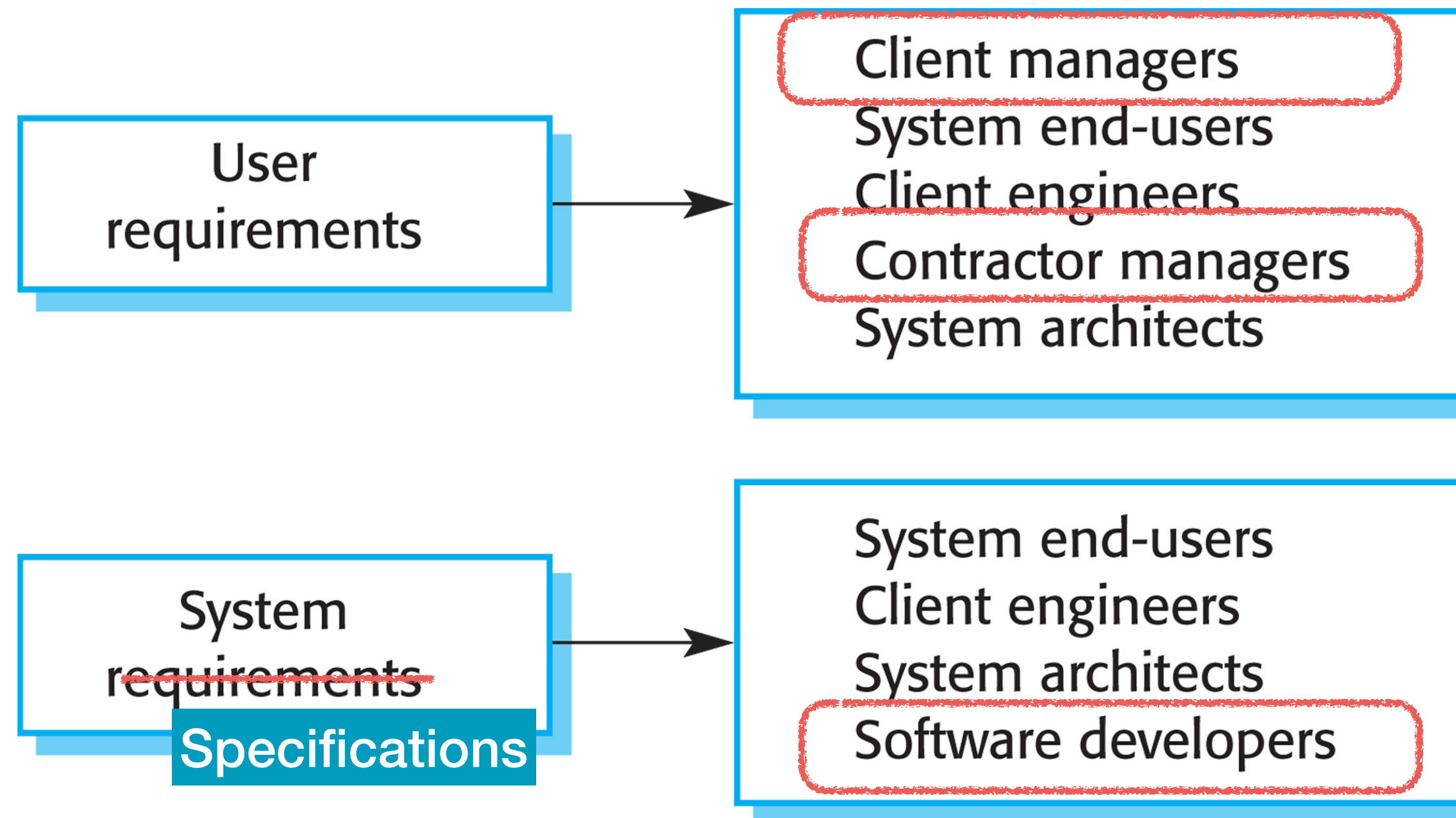
- 1.1 On the last working day of each month, a summary of the drugs prescribed, their cost and the prescribing clinics shall be generated.
- 1.2 The system shall automatically generate the report for printing after 17.30 on the last working day of the month.
- 1.3 A report shall be created for each clinic and shall list the individual drug names, the total number of prescriptions, the number of doses prescribed and the total cost of the prescribed drugs.
- 1.4 If drugs are available in different dose units (e.g. 10mg, 20 mg, etc.) separate reports shall be created for each dose unit.
- 1.5 Access to all cost reports shall be restricted to authorized users listed on a management access control list.

What the system
must do to meet
this user requirement



Who looks at Specifications?

Figure 4.2 Readers of different types of requirements specification



Copyright ©2016 Pearson Education, All Rights Reserved



University of
Nottingham

UK | CHINA | MALAYSIA

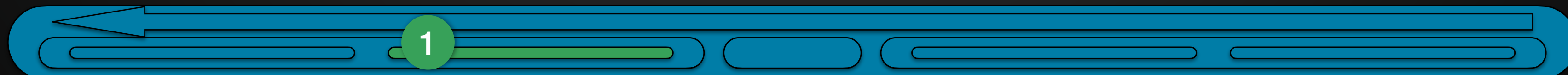
What Specifications are like



Specifications

... detailed descriptions of the software system's functions, services, and operational constraints ... they ... should define **exactly what is to be implemented.**"

- A focus on WHAT should be built - but not necessarily HOW
- Specifying: what a system will do to meet the user requirements
- This means specifications should be traceable to a requirement
 - specifications are often tabulated,
 - with a column saying which requirement ID it is for, etc
 - priorities, risks, etc



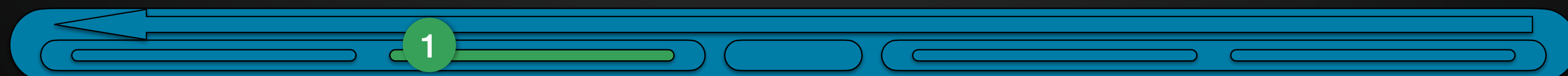
Specifications

Notation	Description
Natural language sentences	The Specifications are written using numbered sentences in natural language. Each sentence should express one req Specification
Structured natural language	The Specifications are written in natural language on a standard form or template. Each field provides information about an aspect of the requirement Specification
Graphical notations	Graphical models, supplemented by text annotations, are used to define the functional Specifications for the system. UML (unified modeling language) use case and sequence diagrams are commonly used.
Mathematical specifications	These notations are based on mathematical concepts such as finite-state machines or sets. Although these unambiguous specifications can reduce the ambiguity in a requirements document, most customers don't understand a formal specification. They cannot check that it represents what they want, and they are reluctant to accept it as a system contract. (I discuss this approach, in Chapter 10, which covers system dependability.)

Copyright ©2016 Pearson Education, All Rights Reserved

Specifications

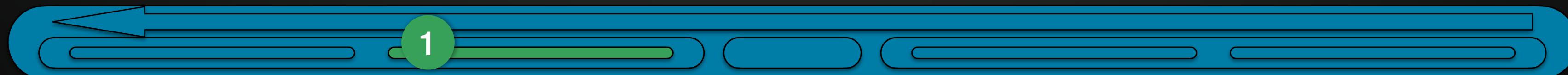
Figure 4.11 Notations for writing system ~~requirements~~





Specs - Natural Language

- Can be expressive, intuitive, and universal
- May therefore be ambiguous, vague, and interpreted differently
- Guidelines
 - 1) Use a standard format: 1 sentence, linked to a user req.
 - 2) Distinguish between mandatory ('shall') and desirable ('should')
 - 3) Emphasis important elements with bold, italic, etc
 - 4) avoid jargon, unless clearly specified in a key words section
 - 5) **make sure the specification is measurable** in some form
 - cant be partly achieved, or has a metric for successfully achieved





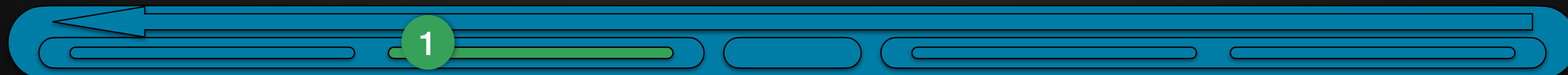
Specs - Natural Language

3.2 The system shall measure the blood sugar and deliver insulin, if required, every 10 minutes. (*Changes in blood sugar are relatively slow, so more frequent measurement is unnecessary; less frequent measurement could lead to unnecessarily high sugar levels.*)

3.6 The system shall run a self-test routine every minute with the conditions to be tested and the associated actions defined in Table 1. (*A self-test routine can discover hardware and software problems and alert the user to the fact the normal operation may be impossible.*)

Copyright ©2016 Pearson Education, All Rights Reserved

Figure 4.12 Example ~~requirements~~ for the insulin pump software system
Specifications





Specifications

- From April 1999 - One project defining their terms

In this document, several words are used to signify the ~~requirements~~ of the specification. These words are often capitalized. Importance

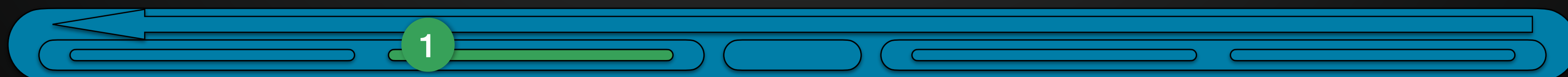
MUST Usually.

MUST NOT Usually not.

SHOULD Only when Marketing insists.

MAY Only if it doesn't cost extra.

- Other formal definitions
 - <https://tools.ietf.org/html/rfc2119>

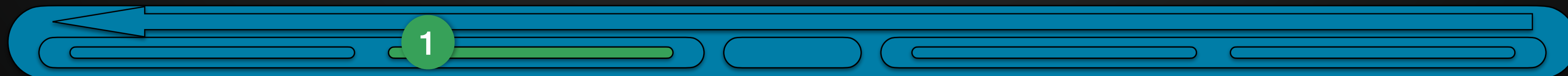




Specs - Structured

*When you need to be **exact**, about e.g. conditions or calculations*

- Go further than natural language specifications, to tabulate specifications, or put them in templates.
 - these enforce certain details
- Can be used to specify additional information
 - associated logic in the function
 - inputs and outputs
 - an explanation
 - conditions
 - side effects or relations to other functions



Specs - Structured

Insulin Pump/Control Software/SRS/3.3.2

Function	Compute insulin dose: Safe sugar level.
Description	Computes the dose of insulin to be delivered when the current measured sugar level is in the safe zone between 3 and 7 units.
Inputs	Current sugar reading (r2), the previous two readings (r0 and r1).
Source	Current sugar reading from sensor. Other readings from memory.
Outputs	CompDose—the dose in insulin to be delivered.
Destination	Main control loop.
Action:	CompDose is zero if the sugar level is stable or falling or if the level is increasing but the rate of increase is decreasing. If the level is increasing and the rate of increase is increasing, then CompDose is computed by dividing the difference between the current sugar level and the previous level by 4 and rounding the result. If the result, is rounded to zero then CompDose is set to the minimum dose that can be delivered. (see Figure 4.14)
Requires	Two previous readings so that the rate of change of sugar level can be computed.
Precondition	The insulin reservoir contains at least the maximum allowed single dose of insulin.
Postcondition	r0 is replaced by r1 then r1 is replaced by r2.
Side effects	None.

Copyright ©2016 Pearson Education, All Rights Reserved

Figure 4.13 *The structured specification of a requirement for an insulin pump*

Specs - Structured - Tabular

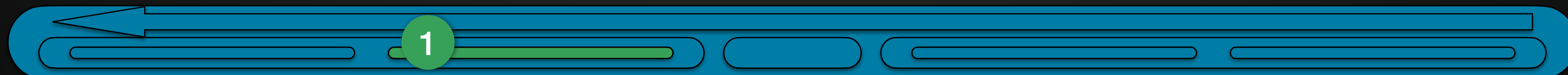
- For e.g. when a spec has options that need explicitly listing

Condition	Action
Sugar level falling ($r2 < r1$)	CompDose = 0
Sugar level stable ($r2 = r1$)	CompDose = 0
Sugar level increasing and rate of increase decreasing ($(r2 - r1) < (r1 - r0)$)	CompDose = 0
Sugar level increasing and rate of increase stable or increasing $r2 > r1$ & $((r2 - r1) \geq (r1 - r0))$	CompDose = round $((r2 - r1)/4)$ If rounded result = 0 then CompDose = MinimumDose

Copyright ©2016 Pearson Education, All Rights Reserved

Figure 4.14 The tabular specification of computation in an insulin pump

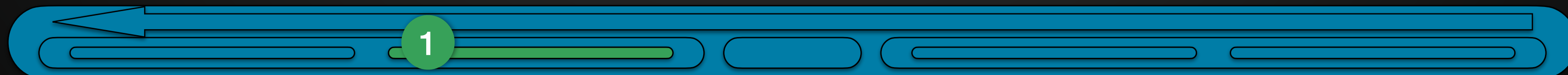
- Often helps with mathematical ones too





Specs - Graphical

- UML Models, diagrams, prototypes
 - » At this stage, though, the emphasis is on saying
 - » ‘this is how it will work’
 - » ‘this is what you “mr developer” should build’
- Its often easier to see a UML sequence diagram
 - » Than to read 30 decision-dependent specifications
- So when you find specifications are complex - visualise them
 - Just as when you found a requirement to be complex
- UML can be used for either
 - and indeed for post-development documentation (as said a few lectures ago)



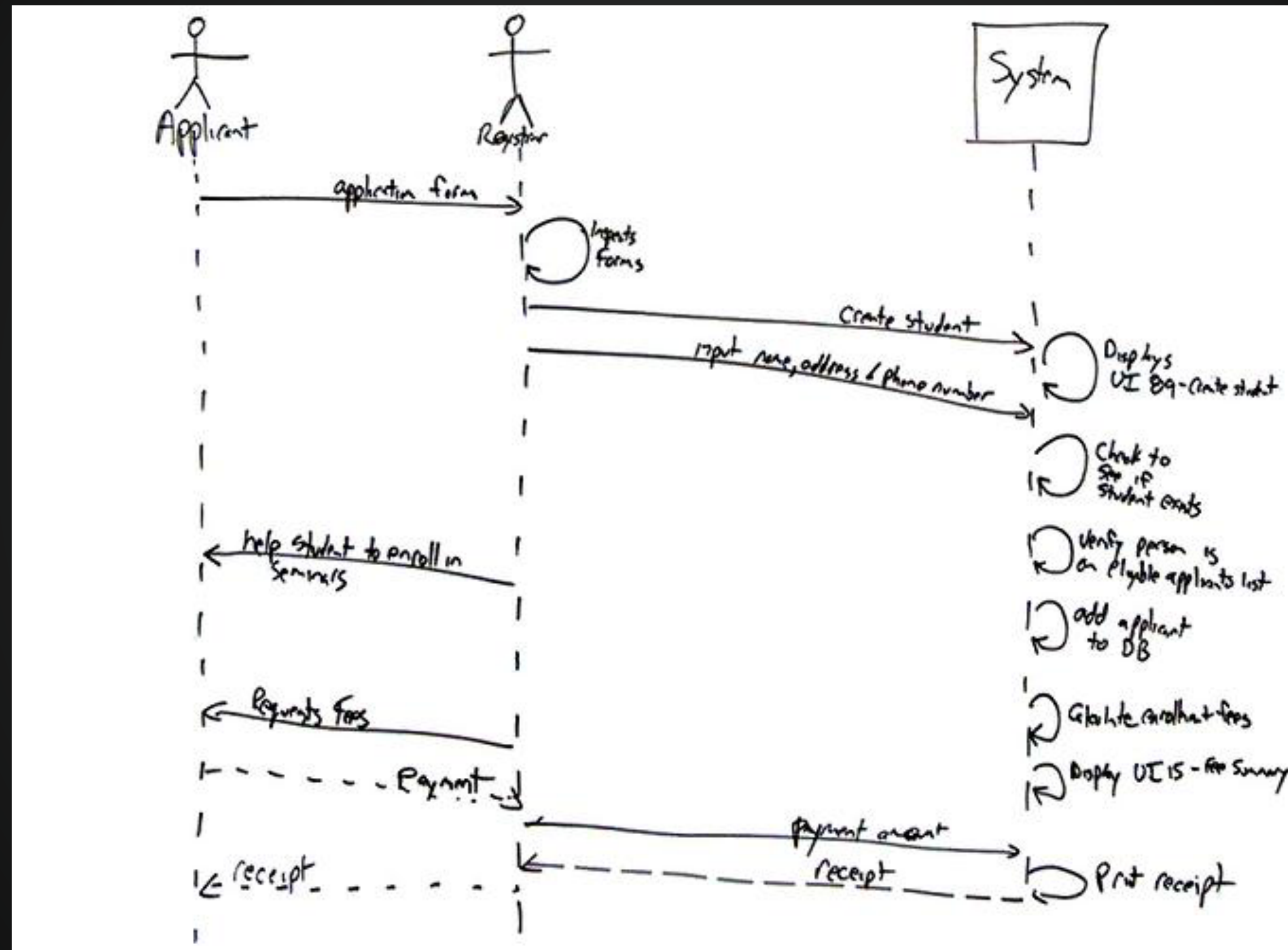


University of
Nottingham

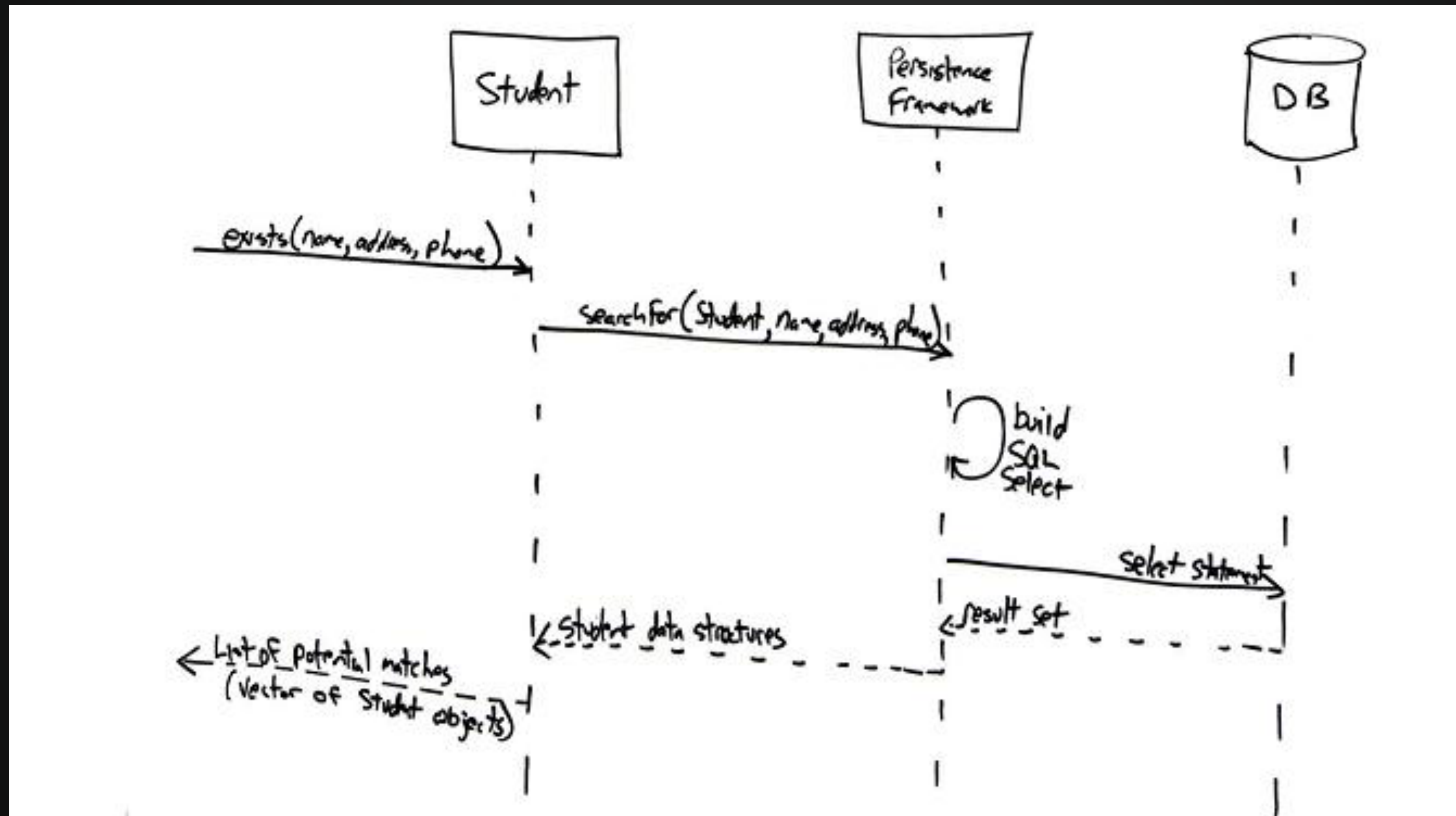
UK | CHINA | MALAYSIA

Difference between Requirements Diagrams and Specifications Diagrams

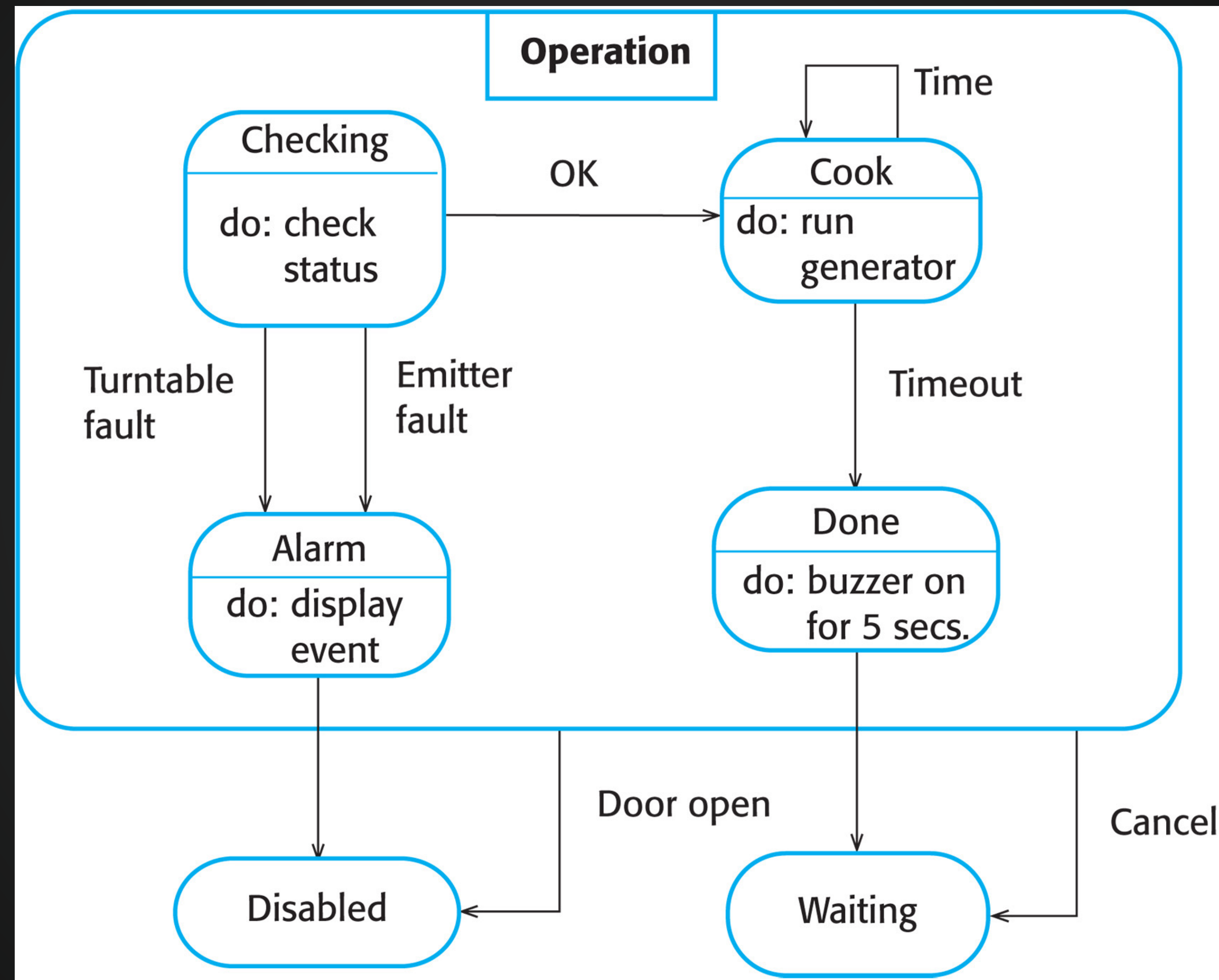
Sequence Diagram - Requirements



Sequence Diagram - Specifications

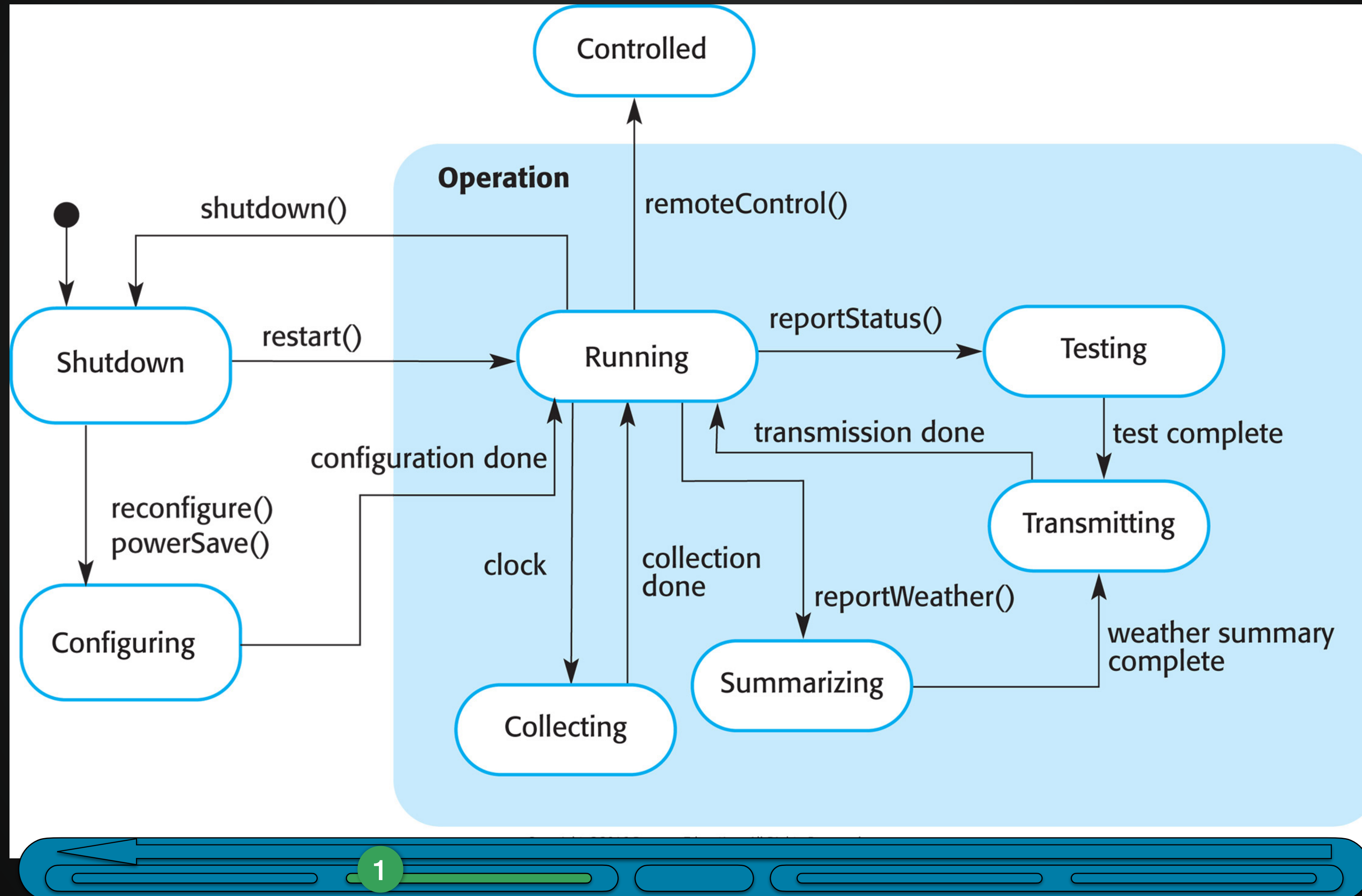


State Diagram - Requirements





State Diagram - Specifications





University of
Nottingham

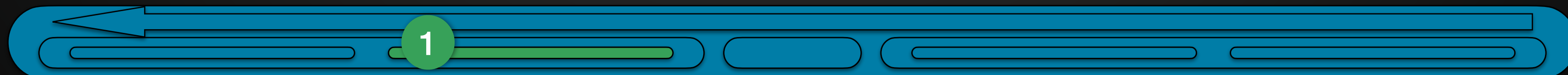
UK | CHINA | MALAYSIA

Good Specifications



Qualities of Good Specifications

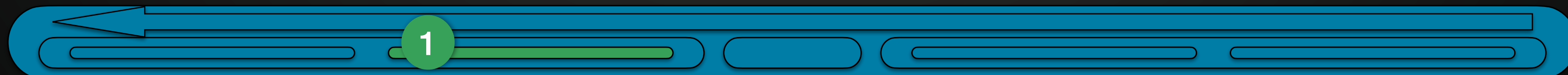
- Analysing the quality of specifications is also important
- Good quality specifications have a few qualities
- Specifications should be
 - Correct –Complete
 - Necessary –Clear
 - Feasible –Consistent
 - Unique –Traceable
 - Concise –Verifiable





Specs - Traceability

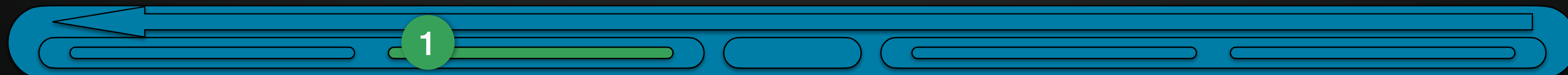
- Its really important that all specifications can be traced to user requirements
- In reports, you should, for every specification
 - state which user requirement(s) it is supporting
- You may also categorise them by importance, difficulty, etc





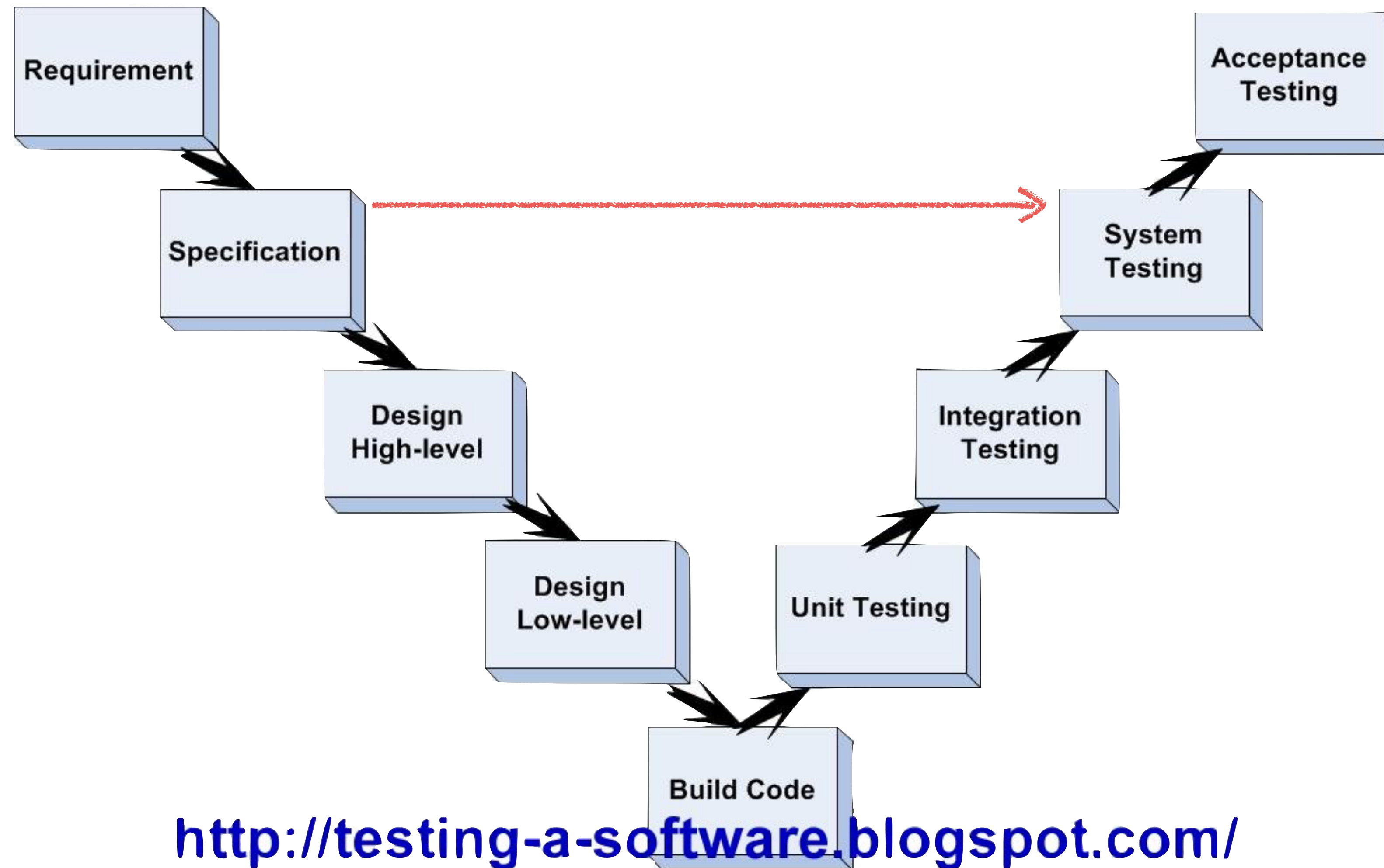
Specs - Testability

- Verification - did we build it right?
- To have implemented a single spec, you'll want to know you achieved what it specified.
 - you can only do this if specifications are testable.
- Can all the things you specify be tested and proved?
 - 'it must load fast' - unprovable
 - 'it must load within 10s' - testable and provable





Specs - Testability

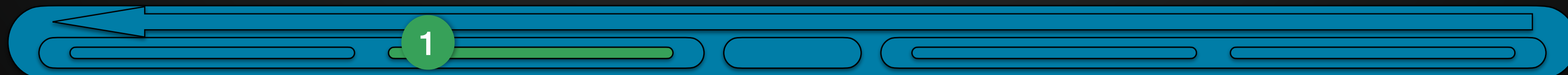




Specification Reviews

“The specifications are analysed systematically by a team of reviewers who **check for errors and inconsistencies**.”

- There is a formal review process you can go through
 - Several people in a room, **reading each Spec aloud**.
- Each person takes a Role - to systematically review the specifications:
 - Validity checks (are the areas of functionality identified as necessary)
 - Consistency checks (do specifications conflict with one another)
 - Completeness checks (does it specify a coherent system or only parts of it)
 - Realism checks (can specifications actually be implemented)
 - Verifiability checks (can specifications be tested)
- Are the system specifications: correct, necessary, important?





Today's Learning Outcomes

- 0) What specifications are (and what they look like)
- 1) How they are different to requirements
- 2) What makes a good/bad specification
- 3) What they are used for
- 4) What they are not good for



Have a great week! Upload Memes!